

Facet ZK Fault Proof Rollup

Security Review

Solo review by:

Slowfi, Security Researcher

August 5, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Gas Optimization	4
3.1.1	Missing Event Emission for L1 Block Hash Caching and Unstructured Duplication . . .	4
3.1.2	Redundant Call to <code>_proposalConflictsWithCanonical()</code> in Resolution Logic	4
3.2	Informational	5
3.2.1	Missing External Dependency: <code>facet-sol</code>	5
3.2.2	Reversed Custom Errors in Game State Modifiers	5
3.2.3	Misleading Comment Suggesting Alternate Canonical Assignment	6
3.2.4	Consider overwriting <code>renounceOwnership()</code> and usage of <code>Ownable2Step</code>	6
3.2.5	Inconsistent Enum Formatting Between <code>ResolutionStatus</code> and <code>ProposalStatus</code> . . .	7
3.2.6	<code>getAnchorRoot()</code> Can Be Marked <code>external</code>	7
3.2.7	ZK-Proven Proposals Can Be Bypassed by Non Proven Optimistic on Canonical Selection	8
3.2.8	Different Timestamp Data Types in <code>Proposal Struct</code>	8

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity	Description
Critical	<i>Must fix as soon as possible (if already deployed).</i>
High	Leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	Global losses <10% or losses to only a subset of users, but still unacceptable.
Low	Losses will be annoying but bearable. Applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.
Gas Optimization	Suggestions around gas saving practices.
Informational	Suggestions around best practices or readability.

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Facet achieves forced inclusion impossible to turn off by making the forced transaction "inbox" a permissionless EOA address instead of a smart contract and give forced transactions a reasonable share of L2 blockspace by enabling all user transactions to have an equal share of L2 blockspace in an "anything goes" based sequencing model.

From Jul 28th to Jul 29th the security researchers conducted a review of [zk-fault-proofs](#) on commit hash [e85de243](#). A total of **10** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Gas Optimizations	2	1	1
Informational	8	5	3
Total	10	6	4

3 Findings

3.1 Gas Optimization

3.1.1 Missing Event Emission for L1 Block Hash Caching and Unstructured Duplication

Severity: Gas Optimization

Context: [Rollup.sol#L412-L419](#)

Description: The logic within `getL1BlockHash()` caches recent L1 block hashes using `blockhash(...)` for blocks in the last 256 range. However, it does not emit the `L1BlockHashCheckpointed` event when a block hash is retrieved and stored:

```
if (l1BlockHash == bytes32(0)) {
    l1BlockHash = blockhash(l1BlockNumber);
    if (l1BlockHash == bytes32(0)) {
        revert L1BlockHashNotCheckpointed();
    }
    l1BlockHashes[l1BlockNumber] = l1BlockHash; // No event emitted
}
```

By contrast, the `checkpointL1BlockHash()` function at line 589 performs the same storage logic and does emit `L1BlockHashCheckpointed`. This leads to an inconsistency: some cached block hashes are tracked via events while others are not, even though both write to the same state variable. Additionally, both functions share duplicated logic for writing the block hash to storage and could benefit from encapsulating the repeated steps into a shared internal function to promote clarity and maintain consistency.

Recommendation: Emit the `L1BlockHashCheckpointed` event inside `getL1BlockHash()` when a block hash is stored. Consider refactoring the shared logic into an internal helper like `_storeL1BlockHash(...)` to eliminate duplication. Example:

```
function _storeL1BlockHash(uint256 l1BlockNumber, bytes32 blockHash) internal {
    l1BlockHashes[l1BlockNumber] = blockHash;
    emit L1BlockHashCheckpointed(l1BlockNumber, blockHash);
}
```

Facet: Fixed in commit [7bdb8b14](#).

Slowfi: Fix verified. Fixed by removing the storage of the checkpoint on `getL1BlockHash` and transforming it into a view function.

3.1.2 Redundant Call to `_proposalConflictsWithCanonical()` in Resolution Logic

Severity: Gas Optimization

Context: [Rollup.sol#L444-L447](#), [Rollup.sol#L512-L515](#)

Description: In `resolveProposal(...)`, the function `_proposalConflictsWithCanonical(p)` is called directly:

```
else if (_proposalConflictsWithCanonical(p)) {
    p.resolutionStatus = ResolutionStatus.CHALLENGER_WINS;
}
```

Then, immediately afterward, `_getProposalProver(p)` is called, which internally repeats the same check:

```
else if (_getProposalProver(p) != address(0)) {
    // ...
}
```

This results in a duplicated call to `_proposalConflictsWithCanonical(p)`.

Recommendation: Consider refactoring the logic to avoid redundant checks and improve gas efficiency. Storing the result in a local variable or restructuring the resolution flow could help simplify the code while preserving correctness.

Facet: Acknowledged.

Slowfi: Acknowledged.

3.2 Informational

3.2.1 Missing External Dependency: facet-sol

Severity: Informational

Context: L1Bridge.sol#L10

Description: (*Out Of Scope issue*). The contract L1Bridge.sol imports LibFacet from the external package facet-sol:

```
import {LibFacet} from "facet-sol/src/utils/LibFacet.sol";
```

However, this dependency is not listed in the project's configuration and must be installed manually via:

```
forge install 0xFacet/facet-sol
```

This may cause build failures or confusion for new developers or auditors setting up the project.

Recommendation: Consider explicitly listing 0xFacet/facet-sol as a required dependency in the foundry.toml or providing setup instructions in the project's README or contributing guide.

Facet: Fixed in commit 1c395b5d.

Slowfi: Fix verified.

3.2.2 Reversed Custom Errors in Game State Modifiers

Severity: Informational

Context: Rollup.sol#L363-L371

Description: The onlyIfGameOver and onlyIfGameNotOver modifiers in Rollup.sol revert with misleading custom errors that contradict their names. Currently:

```
modifier onlyIfGameOver(uint256 proposalId) {
    if (!gameOver(proposalId)) revert GameOver(); // should be GameNotOver
    _;
}

modifier onlyIfGameNotOver(uint256 proposalId) {
    if (gameOver(proposalId)) revert GameNotOver(); // should be GameOver
    _;
}
```

As a result, when a proposal's game has ended, calling a function restricted by onlyIfGameNotOver throws GameNotOver(), even though the game is over. This inversion can lead to confusion during integration, testing, and debugging.

Proof Of Concept:

```
function test_gameOverErrorMismatch() public {
    bytes32 root = bytes32(uint256(1));
    uint256 l2BlockNumber = 1613382 + 1800;
    uint256 parentId = 0;

    // Advance time so proposal will be valid and resolvable
    vm.warp(block.timestamp + 1800 * 180);
    rollup.submitProposal{value: 0.001 ether}(root, l2BlockNumber, parentId);

    // Advance time beyond deadline to ensure game is over
    uint256 ;
    ids[0] = 1;
    Rollup.Proposal[] memory proposals = rollup.getProposals(ids);
    vm.warp(proposals[0].deadline + 1);

    rollup.resolveProposal(1); // Finalize proposal

    assertEq(rollup.gameOver(1), true); // Confirm game is over

    vm.expectRevert(Rollup.GameOver.selector); // Fails reverts with GameNotOver
    rollup.challengeProposal{value: 5 ether}(1);
}
```

Recommendation: Update the modifier logic to correctly match the intended error semantics:

```
modifier onlyIfGameOver(uint256 proposalId) {
    if (!gameOver(proposalId)) revert GameNotOver(); // Clearer
    _;
}

modifier onlyIfGameNotOver(uint256 proposalId) {
    if (gameOver(proposalId)) revert GameOver(); // Clearer
    _;
}
```

This ensures error messages reflect the actual cause of failure and improves clarity for developers and external integrations.

Facet: Fixed in commit [55daecb9](#).

Slowfi: Fix verified.

3.2.3 Misleading Comment Suggesting Alternate Canonical Assignment

Severity: Informational

Context: [Rollup.sol#L457](#)

Description: The following comment in `Rollup.sol` at line 457 suggests that a proposal may already have been marked as canonical by a "validity proof":

```
// Mark as canonical if not already set by validity proof
_trySetCanonical(p.l2BlockNumber, id);
```

However, there is no alternative path in the contract where `_trySetCanonical(...)` is called outside of:

- The constructor (for the genesis proposal), and.
- This exact line in `resolveProposal(...)`.

Even `proveBlock()` calls `_createProposal()` and then immediately calls `resolveProposal(...)`, which hits this same code path. Thus, the comment may lead developers or auditors to incorrectly believe that canonical proposals can be set elsewhere via some implicit mechanism in validity proofs.

Recommendation: Update the comment to more accurately reflect actual behavior or remove the comment entirely if it adds confusion.

Facet: Fixed in commit [55daecb9](#).

Slowfi: Fix verified.

3.2.4 Consider overwriting `renounceOwnership()` and usage of `Ownable2Step`

Severity: Informational

Context: [Rollup.sol#L19](#)

Description: The `Rollup` contract inherits from OpenZeppelin's `Ownable`, which exposes the `renounceOwnership()` function. This allows the current owner to permanently set the owner to the zero address. However, the `setProposer()` function - which manages access control for submitting fault-proof proposals - is restricted to the contract owner:

```
function setProposer(address proposer, bool allowed) external onlyOwner;
```

If `renounceOwnership()` is called, this function becomes permanently unusable. Unless the contract is explicitly designed to transition into a fully decentralized state, this may unintentionally freeze access control logic. Additionally, `Ownable`'s `transferOwnership()` function could be replaced with a safer two-step variant (`Ownable2Step`) to reduce the risk of accidental or malicious ownership transfers.

Recommendation: Consider overriding `renounceOwnership()` to disable or restrict its usage unless explicitly required:

```
function renounceOwnership() public override onlyOwner {
    revert("Renouncing ownership is disabled");
}
```

Alternatively, migrate to `Ownable2Step` from OpenZeppelin to allow safer handover of control. These changes would improve operational safety and reduce the chance of locking critical access control functions.

Facet: Acknowledged.

Slowfi: Acknowledged.

3.2.5 Inconsistent Enum Formatting Between `ResolutionStatus` and `ProposalStatus`

Severity: Informational

Context: [Rollup.sol#L46-L54](#)

Description: The two enums defined in `Rollup.sol` use different formatting styles:

```
// Single-line
enum ResolutionStatus { IN_PROGRESS, DEFENDER_WINS, CHALLENGER_WINS }

// Multi-line
enum ProposalStatus {
    Unchallenged,
    Challenged,
    UnchallengedAndProven,
    ChallengedAndProven,
    Resolved
}
```

Although this does not affect contract functionality, the inconsistency may hinder readability and create unnecessary friction when reviewing or modifying the code.

Recommendation: Consider formatting both enums consistently. Preferably using the multi-line style for better visual clarity and alignment with the rest of the contract.

Facet: Acknowledged.

Slowfi: Acknowledged.

3.2.6 `getAnchorRoot()` Can Be Marked `external`

Severity: Informational

Context: [Rollup.sol#L613](#)

Description: The function `getAnchorRoot()` is currently declared as `public`:

```
function getAnchorRoot() public view returns (bytes32, uint256)
```

However, it is not invoked internally by the contract. Functions that are not used internally and are intended only for external calls can be marked `external` to better communicate intended use.

Recommendation: Update the visibility to `external`:

```
function getAnchorRoot() external view returns (bytes32, uint256)
```

This change is minor but improves clarity.

Facet: Fixed in commit [c6e01504](#).

Slowfi: Fix verified.

3.2.7 ZK-Proven Proposals Can Be Bypassed by Non Proven Optimistic on Canonical Selection

Severity: Informational

Context: [Rollup.sol#L330-L357](#)

Description: Currently, a proposal that passes verification using the `proveProposal()` function is not automatically resolved, even when its parent is already resolved and the game is over. This behavior can lead to a valid and proven proposal being bypassed in favor of an earlier unchallenged one, especially when both are competing for the same L2 block.

- Example scenario:
 1. Parent proposal is confirmed (resolved in favor of defender).
 2. Proposal 1 (optimistic) is submitted shortly after, but unchallenged.
 3. Proposal 2 (ZK) is submitted and proven, but slightly later.
 4. If `resolveProposal` is called on Proposal 1 just before Proposal 2 is resolved, the canonical for that L2 block may be locked to Proposal 1, effectively bypassing the proven proposal.

This is a highly unlikely outcome and direct violation of system intended design.

Recommendation: Consider adding conditional auto-resolution for proposals proven via `proveProposal()` when their parent is already resolved. This would reduce the potential for valid proofs to be bypassed and align better with the system's intended prioritization of ZK validity over optimistic assumptions.

That said, even if this is implemented, the issue may still occur for proposals that are proven ahead of the current anchor (i.e., not the immediate child of the latest resolved block), as resolution must still occur explicitly in those cases. Therefore, this fix would only mitigate and not eliminate the underlying race condition.

Alternatively, clearly documenting this edge case as a known tradeoff in the design would help downstream integrators and verifiers better understand the protocol behavior.

Facet: Provers interested in immediate resolution can always call `proveBlock` instead of `proveProposal`. `proveProposal` does not immediately resolve because it is designed for proving proposals with an unresolved parent, and such proposals cannot be resolved immediately even when proven.

Implementing the suggested fix would only have an impact in the case that an invalid proposal went unchallenged for the entire 7 day challenge window and a prover submitted a conflicting proof of a different proposal in the final 6 hours, but did not choose to either challenge the invalid proposal or use the `proveBlock` method. Weighing this against the added complexity, we decided not to make a change.

Slowfi: Acknowledged by Facet team.

3.2.8 Different Timestamp Data Types in `Proposal` Struct

Severity: Informational

Context: [Rollup.sol#L110-L112](#)

Description: The `Proposal` struct uses both `uint32` and `uint64` types to store time-related fields:

```
uint32  deadline;  
uint64  resolvedAt;
```

This discrepancy can be confusing for future maintainers and increases the cognitive overhead when reasoning about the contract's time-related logic. Since both fields represent Unix timestamps, using a consistent type would improve code clarity.

Recommendation: Consider standardizing on a single data type for all timestamp-related fields within the `Proposal` struct either `uint64` for future-proofing or `uint32` if tight packing and space efficiency are critical and overflow is not a concern in the contract's expected lifetime.

Facet: Fixed in commit [5bdd9147](#).

Slowfi: Fix verified.